

# AWS Identity and Access Management (IAM) – Student Handout

## Session Objective

Understand IAM's role in AWS security, key components, access control models, and best practices.

## 1. IAM Overview

- AWS service for secure access management to resources.
- Core Components: Users, Groups, Roles, Policies, Permissions.
- Why IAM Matters:
  - Security: Control who accesses resources.
  - Compliance: Meet audit and regulatory requirements.
  - Least Privilege: Minimize exposure.
- Shared Responsibility: You manage identities and permissions.

## 2. IAM Core Components

**Users:** Permanent identities with credentials.

**Groups:** Logical collection of users.

**Roles:** Temporary credentials assumed by users/services.

**Policies:** JSON permissions documents.

**MFA:** Adds an extra security layer.

## 3. IAM Policy Structure

Elements:

- Version: Policy language version.
- Statement: List of permissions.
- Effect: Allow/Deny.
- Action: What API operations are permitted.
- Resource: ARNs affected.
- Condition: Optional filters (IP, time, MFA, tags).

**Example:**

```
{  
  "Effect": "Allow",
```

```
"Action": "s3:GetObject",  
"Resource": "arn:aws:s3:::bucket/*"  
}
```

## 4. Policy Types

- Identity-Based: Attached to users, groups, or roles.
- Resource-Based: Attached directly to resources (S3, Lambda).
- Session Policy: Temporary restriction.
- Permission Boundary: Sets maximum permission limit.
- SCP: Organization-wide governance.

## 5. Permission Boundaries

- Restricts maximum permissions (no grants).
- Useful for delegated administration.
- Effective Permission = Identity Policy  $\cap$  Boundary.
- Apply to users and roles only.

**Best Practice:** Always apply boundaries for developer-managed roles.

## 6. Least Privilege Principle

- Grant only required permissions.
- Review CloudTrail to remove unused access.
- Use IAM Access Analyzer and Policy Simulator.
- Ongoing process, not one-time setup.

## 7. Federation & Identity Center

- Federation allows external users (AD, Okta, Azure AD) to access AWS.
- Benefits: Centralized management, SSO, no long-lived keys.
- AWS Identity Center (SSO): Multi-account unified login, integrates with IdPs, supports ABAC.

## 8. Cross-Account Access

- Account A hosts resources, Account B needs access.
- Trust Policy (who) + Permission Policy (what).
- Use External IDs, MFA, and monitor with CloudTrail.

## 9. RBAC vs ABAC

**RBAC:** Role-based (Admin, Developer). Simple but limited.

**ABAC:** Attribute-based (tags like Department=Finance). Scalable.

**Hybrid:** Use RBAC for broad access, ABAC for fine-grained control.

## 10. CI/CD Security & OIDC

- Avoid hardcoded AWS keys in pipelines.
- Use OIDC federation for GitHub/GitLab.
- JWT tokens auto-expire (~1 hr).
- Steps:
  1. Create OIDC provider (token.actions.githubusercontent.com)
  2. Create IAM role with trust policy.
  3. Add audience/branch conditions.
  4. Use aws-actions/configure-aws-credentials.

## 11. CI/CD Role Design

- Build Role: S3 read/write, ECR push.
- Deploy Role: CloudFormation, ECS.
- Infra Role: EC2, VPC provisioning.
- Security Scan Role: Read-only for compliance.
- Use per-environment separation and approvals.

## 12. CI/CD Security Checklist

- ✓ Use OIDC (no long-lived keys)
- ✓ MFA for manual ops
- ✓ Session  $\leq$  1 hour
- ✓ Enable CloudTrail logs
- ✓ Branch-based deployment
- ✓ Monthly permission audits
- ✓ Secrets scanning (Dependabot, Snyk)
- ✓ Document all roles

**References:** AWS Docs, GitHub Hardening Guide, OWASP CI/CD Top 10.

## **Summary**

IAM is the foundation of AWS Security.

- Apply least privilege and audit regularly.
- Use federation and temporary credentials.
- Enforce guardrails with SCPs & Boundaries.
- Secure pipelines via OIDC.